

# NeuralBridge Platform

## Technical Reference, Architecture Guide & Operational Handbook

Version 3.4.1 | Classification: Internal Engineering | Document ID: NBP-TRD-2024-09

Prepared by: NeuralBridge Engineering Team | Reviewed by: Platform Architecture Council

Last Updated: September 2024 | Next Review: March 2025

This document is the authoritative technical reference for the NeuralBridge Platform — a distributed, multi-modal AI orchestration system designed for enterprise-scale intelligent document processing, real-time analytics, and adaptive workflow automation. It covers system architecture, component specifications, API contracts, deployment procedures, performance benchmarks, security controls, and operational runbooks.

All engineers, platform operators, and integration architects working with NeuralBridge are expected to be familiar with the contents of this document before making changes to production configurations or initiating new integration projects.

# Table of Contents

---

- 1. Executive Summary & Platform Overview ..... pg. 3
- 2. Core Architecture — The Octane Engine ..... pg. 4
- 3. Data Ingestion Pipeline ..... pg. 6
- 4. Vector Store & Embedding Strategy ..... pg. 8
- 5. Retrieval System — Hybrid Dense + Sparse ..... pg. 10
- 6. Fusion & Reranking Layer ..... pg. 13
- 7. LLM Orchestration & Response Generation ..... pg. 15
- 8. Multi-Modal Processing Module ..... pg. 18
- 9. Security Architecture & Access Controls ..... pg. 20
- 10. Performance Benchmarks & Latency Targets ..... pg. 22
- 11. Deployment — Kubernetes & Helm ..... pg. 24
- 12. API Reference ..... pg. 27
- 13. Monitoring, Observability & Alerting ..... pg. 30
- 14. Known Limitations & Mitigations ..... pg. 32
- 15. Changelog & Version History ..... pg. 34

# 1. Executive Summary & Platform Overview

NeuralBridge is a production-grade AI orchestration platform originally developed as an internal tool at Meridian Labs in 2021 and open-sourced under the Apache 2.0 license in Q1 2023. It is now maintained by a consortium of 14 contributing organizations and has been deployed in over 200 enterprise environments across finance, healthcare, legal, and manufacturing sectors.

The platform's central value proposition is its ability to make large, heterogeneous document corpora queryable with natural language — returning precise, cited, hallucination-resistant answers at sub-500ms latency for 95th percentile queries on corpora of up to 50 million document chunks. This is achieved through a proprietary four-stage retrieval and reasoning pipeline built on top of industry-standard components.

## 1.1 Four Pillars of the Platform

| Pillar                  | Description                                                                   | Primary Benefit                                                     |
|-------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Hybrid Retrieval        | Dense vector search (HNSW) combined with sparse BM25 keyword matching         | BM25 keyword matching improves exact-term matching — recall ↑ 12.5% |
| Reciprocal Rank Fusion  | Merges ranked lists from dense and sparse retrievers                          | Outperforms single retrievers by 18.34% on NDCG@10                  |
| Cross-Encoder Reranking | Top-N candidates from fusion are re-scored by a fine-tuned cross-encoder      | Precision@5 improvement of up to 41% over bi-encoder reranking      |
| Grounded Generation     | LLM response is constrained to retrieved context with a hallucination penalty | Hallucination rate below 0.8% on adversarial evaluation suite       |

## 1.2 Supported Deployment Modes

| Mode            | Description                                   | Typical Use Case                       |
|-----------------|-----------------------------------------------|----------------------------------------|
| Cloud-Managed   | Fully hosted on NeuralBridge Cloud (AWS/GCP)  | SaaS customers, rapid onboarding       |
| Self-Hosted K8s | Helm chart deployment on customer Kubernetes  | Enterprise data-residency requirements |
| Edge Lite       | Quantized models + SQLite vector store        | Air-gapped or IoT environments         |
| Hybrid          | Cloud orchestration + on-premise vector store | Regulated industries (finance, health) |

**Note:** Edge Lite mode supports corpora up to 500,000 chunks only. For larger corpora, self-hosted or cloud-managed modes must be used. Cross-encoder reranking is disabled in Edge Lite to meet memory constraints.

## 2. Core Architecture — The Octane Engine

The Octane Engine is NeuralBridge's central processing unit — a stateless, horizontally scalable orchestration layer that coordinates all retrieval, fusion, reranking, and generation steps for every incoming query. It is written in Go (version 1.21) with Python sidecar services for ML-heavy operations (embedding generation, cross-encoder inference, LLM calls).

The engine is designed around an event-driven pipeline where each stage produces a typed artifact consumed by the next. This design allows individual stages to be replaced, scaled independently, or bypassed via feature flags — making it possible to A/B test retrieval strategies without redeployment.

### 2.1 Component Map

| Component          | Language | Port | Responsibility                                                     |
|--------------------|----------|------|--------------------------------------------------------------------|
| Query Gateway      | Go       | 8080 | Request validation, auth, rate limiting, query normalization       |
| Embedding Service  | Python   | 8081 | Text-to-vector conversion using configurable embedding model       |
| Dense Retriever    | Go       | 8082 | HNSW index search via Qdrant client; top-K ANN lookup              |
| Sparse Retriever   | Go       | 8083 | BM25 inverted index search; term frequency scoring                 |
| Fusion Service     | Go       | 8084 | RRF merge of dense and sparse ranked lists                         |
| Reranker Service   | Python   | 8085 | Cross-encoder scoring of fused candidate chunks                    |
| Context Assembler  | Go       | 8086 | Chunk deduplication, context window packing, citation tagging      |
| LLM Gateway        | Go       | 8087 | Prompt construction, LLM API routing, streaming, retry logic       |
| Response Validator | Go       | 8088 | Hallucination detection, confidence scoring, citation verification |
| Telemetry Exporter | Go       | 9090 | Prometheus metrics export, trace generation (OpenTelemetry)        |

### 2.2 Request Lifecycle — Step by Step

Understanding the complete lifecycle of a query through the Octane Engine is essential for debugging latency issues, configuring retrieval parameters, and interpreting telemetry data. The following describes the canonical request path for a standard text query against a pre-ingested document corpus.

| Step | Stage               | Input                    | Output                            | Typical Latency               |
|------|---------------------|--------------------------|-----------------------------------|-------------------------------|
| 1    | Query Normalization | Raw user query string    | Cleaned, tokenized query          | < 2ms                         |
| 2    | Embedding           | Normalized query text    | 768-dim query vector              | 15–40ms (GPU), 80–200ms (CPU) |
| 3    | Dense Retrieval     | Query vector             | Top-200 chunks with cosine scores | 10–30ms (Qdrant HNSW)         |
| 4    | Sparse Retrieval    | Tokenized query          | Top-200 chunks with BM25 scores   | 5–15ms (inverted index)       |
| 5    | RRF Fusion          | Two ranked lists         | Unified top-100 chunk list        | < 3ms                         |
| 6    | Cross-Encoder       | Top-40 query+chunk pairs | Reranked top-20 chunks            | 40–120ms (GPU batch)          |
| 7    | Context Assembly    | Top-20 reranked chunks   | Packed context string + citations | < 5ms                         |
| 8    | LLM Generation      | System prompt + context  | Draft answer with citations       | 300–2000ms (model-dependent)  |

| Step | Stage      | Input                 | Output                        | Typical Latency |
|------|------------|-----------------------|-------------------------------|-----------------|
| 9    | Validation | Draft answer + chunks | Validated answer + confidence | < 10ms          |

**Note:** Steps 3 and 4 (Dense and Sparse Retrieval) execute in parallel. The Fusion Service waits for both to complete before proceeding. The combined P95 latency for steps 3+4+5 is typically under 40ms.

### 3. Data Ingestion Pipeline

Before any document can be retrieved by the Octane Engine, it must pass through the NeuralBridge ingestion pipeline — a multi-stage processing chain that transforms raw source files into indexed, searchable chunks stored in both the vector store and the inverted index. The ingestion pipeline supports 23 file formats natively and can be extended via custom parser plugins.

#### 3.1 Supported Input Formats

| Category        | Formats                                             | Parser                              |
|-----------------|-----------------------------------------------------|-------------------------------------|
| Documents       | PDF, DOCX, PPTX, ODT, RTF                           | Apache Tika + custom post-processor |
| Structured Data | CSV, JSON, JSONL, Parquet, Excel (.xlsx)            | Pandas-based tabular parser         |
| Web Content     | HTML, MHTML, Markdown, reStructuredText             | BeautifulSoup + markdown-it         |
| Code            | Python, JavaScript, TypeScript, Go, Java, SQL, YAML | Tree-sitter AST parser              |
| Audio/Video     | MP3, MP4, WAV (transcription only)                  | Whisper v3 transcription + chunker  |
| Email           | EML, MSG, MBOX archives                             | email.parser + thread reconstructor |
| Images          | PNG, JPEG, TIFF (OCR)                               | Tesseract 5.0 + layout analysis     |
| Databases       | PostgreSQL, MySQL, SQLite (schema + data snapshots) | JDBC connector + schema narrator    |

#### 3.2 Chunking Strategies

Chunking is one of the most critical decisions in any RAG system — it determines the granularity at which information is stored and retrieved. NeuralBridge offers four chunking strategies, each optimized for different content types and retrieval patterns.

| Strategy          | Description                                                          | Best For                           | Chunk Size Default             |
|-------------------|----------------------------------------------------------------------|------------------------------------|--------------------------------|
| Fixed Token       | Splits text into fixed-size windows with configurable overlap        | Human-generated prose documents    | 512 tokens, 64 overlap         |
| Semantic Sentence | Splits on sentence boundaries; groups semantically related sentences | Research papers, reports           | 3–7 sentences per chunk        |
| Hierarchical      | Preserves document structure (section > sub-section > paragraphs)    | Technical manuals, legal documents | 1024 tokens, Child: 256 tokens |
| Code-Aware        | Uses AST boundaries (functions, classes, methods) for code files     | Source code repositories           | 1 function/class per chunk     |

**Important:** Fixed Token chunking can split sentences and even words mid-stream if the token boundary falls inside a word. For prose documents, Semantic Sentence chunking is strongly recommended to preserve information coherence. Fixed Token is acceptable for tabular or structured data where sentence structure is not meaningful.

#### 3.3 Metadata Extraction & Tagging

Every chunk produced by the ingestion pipeline is enriched with a metadata envelope that is stored alongside the vector and text. This metadata is used for filtering during retrieval, for citation generation in responses, and for corpus management operations.

| Metadata Field  | Type     | Source                | Example Value             |
|-----------------|----------|-----------------------|---------------------------|
| doc_id          | UUID     | System-generated      | a3f7b2c1-4d8e-...         |
| chunk_id        | UUID     | System-generated      | f8a1d3e2-7c4b-...         |
| source_file     | string   | Ingestion input       | annual_report_2023.pdf    |
| file_type       | enum     | Parser detection      | pdf                       |
| page_number     | integer  | Parser extraction     | 14                        |
| section_title   | string   | Header detection      | Revenue Analysis Q3       |
| created_at      | ISO8601  | Ingestion timestamp   | 2024-03-15T10:22:05Z      |
| language        | BCP-47   | langdetect library    | en                        |
| word_count      | integer  | Token counter         | 347                       |
| embedding_model | string   | Config at ingest time | text-embedding-3-large    |
| custom_tags     | string[] | User-provided         | ['finance', 'Q3', 'APAC'] |

### 3.4 Ingestion Throughput & Scaling

| Deployment      | Ingestion Rate          | Embedding Throughput  | Notes                      |
|-----------------|-------------------------|-----------------------|----------------------------|
| Single Node     | ~500 pages/min          | ~2,000 chunks/min     | CPU embedding; no GPU      |
| GPU Node (A10)  | ~3,000 pages/min        | ~18,000 chunks/min    | CUDA-accelerated embedding |
| Cluster (8 GPU) | ~22,000 pages/min       | ~140,000 chunks/min   | Distributed worker pool    |
| Cloud-Managed   | Elastic, up to 200k ppm | Up to 1.2M chunks/min | Auto-scales to demand      |

## 4. Vector Store & Embedding Strategy

NeuralBridge uses Qdrant as its primary vector store for production deployments, with pgvector (PostgreSQL extension) supported as an alternative for organizations that prefer relational infrastructure. For Edge Lite deployments, a custom SQLite-backed store called NanoVec is used.

### 4.1 Supported Embedding Models

The embedding model is the single most impactful configuration choice in a RAG system — it determines the semantic representation of all stored content and must match the model used at query time. Changing the embedding model requires full re-ingestion of the corpus.

| Model                  | Dimensions | Max Tokens | License     | Recommended For                                  |
|------------------------|------------|------------|-------------|--------------------------------------------------|
| text-embedding-3-large | 3072       | 8191       | Proprietary | Highest accuracy; general-purpose English        |
| text-embedding-3-small | 1536       | 8191       | Proprietary | Cost-efficient; near-large accuracy at 1/5 cost  |
| text-embedding-ada-002 | 1536       | 8191       | Proprietary | Legacy; still supported, not recommended for new |
| bge-large-en-v1.5      | 1024       | 512        | MIT         | On-premise; strong English performance           |
| bge-m3                 | 1024       | 8192       | MIT         | Multilingual; 100+ languages supported           |
| e5-mistral-7b-instruct | 4096       | 32768      | MIT         | Long-context documents; instruction-following    |
| NB-Embed-v2 (custom)   | 768        | 4096       | Proprietary | NeuralBridge fine-tuned; domain-adapted          |
| nomic-embed-text-v1.5  | 768        | 8192       | Apache 2.0  | Open-source; strong code + prose performance     |

**Note:** NB-Embed-v2 is NeuralBridge's proprietary embedding model fine-tuned on a mixture of enterprise document corpora (legal, finance, engineering). It outperforms bge-large-en-v1.5 by 6.2 points on the BEIR benchmark while using 26% fewer dimensions. It is the default model for new deployments as of NeuralBridge version 3.2.0.

### 4.2 HNSW Index Configuration

NeuralBridge uses Hierarchical Navigable Small World (HNSW) graphs as the approximate nearest neighbor (ANN) index structure within Qdrant. HNSW provides an excellent trade-off between search accuracy (recall) and query latency, and its performance can be tuned through two primary parameters: `m` (the number of bi-directional links per node) and `ef_construction` (the size of the candidate list during index construction).

| Parameter           | Default | Range   | Effect of Increasing                     | Effect of Decreasing                       |
|---------------------|---------|---------|------------------------------------------|--------------------------------------------|
| m                   | 16      | 4–64    | Higher recall, more memory, slower build | Lower recall, less memory, faster build    |
| ef_construction     | 200     | 50–2000 | Better index quality, slower ingestion   | Faster ingestion, lower quality            |
| ef (search)         | 128     | 10–500  | Higher recall, higher query latency      | Lower recall, lower latency                |
| full_scan_threshold | 10000   | 1k–100k | Fewer full scans triggered               | More full scans for small filtered results |

### 4.3 Recommended Index Profiles

| Profile     | m  | ef_construction | ef (search) | Use Case                                 |
|-------------|----|-----------------|-------------|------------------------------------------|
| High Recall | 32 | 400             | 256         | Production QA systems; accuracy critical |



| Profile        | m  | ef_construction | ef (search) | Use Case                                 |
|----------------|----|-----------------|-------------|------------------------------------------|
| Balanced       | 16 | 200             | 128         | Default; suits most enterprise workloads |
| Low Latency    | 8  | 100             | 64          | Real-time chat; latency < 20ms required  |
| Bulk Ingestion | 12 | 80              | 100         | Initial corpus loading; rebuild after    |

## 5. Retrieval System — Hybrid Dense + Sparse

NeuralBridge's retrieval system is the core differentiator of the platform. Rather than relying on a single retrieval mechanism, it runs dense vector retrieval and sparse BM25 retrieval in parallel, then merges the results. This hybrid approach captures different complementary signals and consistently outperforms either method in isolation across all tested domains.

### 5.1 Dense Retrieval — How It Works

Dense retrieval operates in the continuous embedding space. The query is converted to a vector by the same embedding model used during ingestion. The HNSW index is then searched using approximate nearest neighbor algorithms to find the top-K document chunks whose embeddings are most similar to the query embedding, measured by cosine similarity.

Dense retrieval excels at capturing semantic relationships — synonyms, paraphrases, conceptual similarity, and cross-lingual matches (when using a multilingual embedding model). It struggles with queries that contain rare proper nouns, product codes, version numbers, or other high-specificity terms that may not have been well-represented in the embedding model's pre-training data.

| Aspect            | Dense Retrieval Behavior                                                                 |
|-------------------|------------------------------------------------------------------------------------------|
| Similarity Metric | Cosine similarity between query vector and chunk vectors                                 |
| Index Structure   | HNSW graph in Qdrant                                                                     |
| Top-K Default     | 200 candidates returned to fusion layer                                                  |
| Filtering         | Metadata pre-filters applied before ANN search (payload filtering in Qdrant)             |
| Strengths         | Semantic similarity, paraphrase matching, multi-lingual, long-context queries            |
| Weaknesses        | Rare terms, exact product codes, version strings, numbers, acronyms not in training data |
| Latency (P95)     | 28ms on 10M chunk corpus with GPU Qdrant, m=16, ef=128                                   |

### 5.2 Sparse Retrieval — BM25 Implementation

Sparse retrieval uses BM25 (Best Match 25), the industry-standard probabilistic term-frequency weighting model. NeuralBridge maintains a separate inverted index built on Apache Lucene (via OpenSearch) that is updated in real-time as documents are ingested. BM25 scoring rewards chunks that contain the query terms frequently relative to how common those terms are across the entire corpus.

#### BM25 Scoring Formula:

```
BM25(q, d) = SUM over terms t in q of:
  IDF(t) * [ tf(t,d) * (k1 + 1) ] / [ tf(t,d) + k1 * (1 - b + b * |d|/avgdl) ]

Where: tf(t,d) = term frequency of t in document d
      IDF(t) = log[(N - df(t) + 0.5) / (df(t) + 0.5) + 1]
      |d| = length of document d in words
      avgdl = average document length in corpus
      k1 = term saturation parameter (default: 1.2)
      b = length normalization parameter (default: 0.75)
```

| BM25 Parameter | Default | Effect of Higher Value                             | Effect of Lower Value                         |
|----------------|---------|----------------------------------------------------|-----------------------------------------------|
| k1             | 1.2     | More reward for repeated terms (saturation higher) | Quickly saturates; frequency matters less     |
| b              | 0.75    | Stronger length normalization                      | Shorter docs rank higher regardless of length |

Sparse retrieval excels precisely where dense retrieval fails: exact term matching for product codes, version numbers, proper nouns, technical identifiers, and rare domain-specific terms. It fails on semantic queries where the query and the answer use completely different vocabulary.

### 5.3 Metadata Filtering

Both dense and sparse retrievers support metadata pre-filtering, which restricts the search space to chunks that match specified conditions before ranking. Filtering is applied as a payload condition in Qdrant (for dense) and as a Lucene query filter (for sparse).

| Filter Type  | Syntax Example                                                 | Supported Operators    |
|--------------|----------------------------------------------------------------|------------------------|
| Equality     | <code>{'file_type': 'pdf'}</code>                              | must, should, must_not |
| Range        | <code>{'created_at': {'gte': '2024-01-01'}}</code>             | gte, lte, gt, lt       |
| Keyword list | <code>{'custom_tags': {'any': ['finance', 'Q3']}}</code>       | any, all               |
| Full-text    | <code>{'section_title': {'match': {'text': 'Revenue'}}}</code> | Lucene match query     |
| Nested       | Boolean combinations of the above                              | AND, OR, NOT logic     |

**Important:** Aggressive metadata filtering on the dense retriever can significantly reduce the effective search space, causing the HNSW index to fall back to full-scan mode when fewer than `full_scan_threshold` candidates remain after filtering. Monitor the `'dense_retriever_full_scan_ratio'` Prometheus metric if filtering is causing latency spikes.

### 5.4 Retrieval Parameter Reference

| Parameter           | Config Key                            | Default | Description                                                                          |
|---------------------|---------------------------------------|---------|--------------------------------------------------------------------------------------|
| Dense Top-K         | <code>retrieval.dense.top_k</code>    | 200     | Candidates from dense retriever to fusion                                            |
| Sparse Top-K        | <code>retrieval.sparse.top_k</code>   | 200     | Candidates from sparse retriever to fusion                                           |
| Dense Weight        | <code>retrieval.dense.weight</code>   | 0.6     | Relative weight in weighted score merge (alternative to <code>fusion_weight</code> ) |
| Sparse Weight       | <code>retrieval.sparse.weight</code>  | 0.4     | Relative weight for sparse in score merge                                            |
| Enable Hybrid       | <code>retrieval.hybrid.enabled</code> | true    | Set false to use dense-only mode                                                     |
| Min Score Threshold | <code>retrieval.min_score</code>      | 0.0     | Discard chunks below this similarity score                                           |
| Timeout             | <code>retrieval.timeout_ms</code>     | 500     | Max ms to wait for retrieval before fallback                                         |

## 6. Fusion & Reranking Layer

The fusion and reranking layer is where NeuralBridge's retrieval quality advantage is most pronounced. After dense and sparse retrievers independently return their top-K candidate chunks, the Fusion Service merges these lists using Reciprocal Rank Fusion. The fused list then passes to the Reranker Service, which uses a fine-tuned cross-encoder to produce the final ranked candidates for context assembly.

### 6.1 Reciprocal Rank Fusion (RRF)

Reciprocal Rank Fusion is a rank aggregation algorithm that combines multiple ranked lists without requiring the scores from different systems to be on a comparable scale. This makes it ideal for merging dense (cosine similarity, range 0–1) and sparse (BM25 score, unbounded) retrievers, since RRF only uses the rank position, not the raw score.

#### RRF Formula:

$$\text{RRF\_score}(d) = \text{SUM over ranked list } r \text{ of: } 1 / (k + \text{rank\_r}(d))$$

Where:  $d$  = a document chunk

$r$  = one of the ranked lists (dense, sparse)

$\text{rank\_r}(d)$  = position of  $d$  in list  $r$  (1-indexed)

$k$  = smoothing constant (NeuralBridge default: 60)

If  $d$  not present in list  $r$ : contribution is 0

The  $k$  constant controls how steeply the contribution of lower-ranked documents falls off. A higher  $k$  gives more weight to documents ranked lower in any individual list (more lenient). A lower  $k$  concentrates weight on top-ranked documents (more aggressive). The value  $k=60$  was originally proposed by Cormack et al. (2009) and has proven robust across most NeuralBridge production deployments.

| k Value   | Behavior                                             | Recommended When                                 |
|-----------|------------------------------------------------------|--------------------------------------------------|
| $k = 10$  | Top-1 chunk gets ~10x the weight of rank-10 chunk    | Very high quality retrievers; small corpora      |
| $k = 60$  | Moderate decay; standard default                     | General production use; mixed-quality retrievers |
| $k = 100$ | Slow decay; many low-rank documents still contribute | Diverse corpora; retrieval uncertainty high      |

### 6.2 Cross-Encoder Reranking

After RRF fusion, the top-N candidates (default: 40) are passed to the Reranker Service. Unlike the bi-encoder used for dense retrieval — which encodes query and document independently and compares them in vector space — the cross-encoder processes the query and each candidate chunk together as a single concatenated input, allowing full self-attention across both.

This joint encoding makes cross-encoders dramatically more accurate than bi-encoders for relevance judgment, but also much slower — typically 5–20ms per query-chunk pair. This is why reranking is only applied to the top-N fused candidates rather than the full retrieval set.

| Cross-Encoder Model     | Base Model  | Latency/pair | NDCG@10 (MS-MARCO) | License    |
|-------------------------|-------------|--------------|--------------------|------------|
| ms-marco-MiniLM-L-6-v2  | MiniLM-L-6  | 2.1ms        | 0.384              | Apache 2.0 |
| ms-marco-MiniLM-L-12-v2 | MiniLM-L-12 | 4.3ms        | 0.392              | Apache 2.0 |

| Cross-Encoder Model          | Base Model   | Latency/pair  | NDCG@10 (MS-MARCO)   | License     |
|------------------------------|--------------|---------------|----------------------|-------------|
| bge-reranker-large           | BERT-Large   | 11.2ms        | 0.411                | MIT         |
| bge-reranker-v2-m3           | BGE-M3       | 14.8ms        | 0.426 (multilingual) | MIT         |
| NB-Reranker-v3 (proprietary) | DeBERTa-v3-L | 18.4ms        | 0.441                | Proprietary |
| Cohere Rerank 3.5 (API)      | Cohere       | API-dependent | 0.451                | Commercial  |

**Note:** NB-Reranker-v3 is the default reranker in NeuralBridge Cloud and self-hosted deployments with GPU. In CPU-only deployments, ms-marco-MiniLM-L-6-v2 is the default due to its 2.1ms/pair latency. Cohere Rerank 3.5 requires an external API call and adds network latency; it is recommended only when maximum accuracy is required and latency SLAs allow.

## 6.3 Reranker Top-N Configuration

The `reranker_top_n` parameter (config key: `reranking.top_n`) controls how many fused candidates are sent to the cross-encoder. Higher values improve recall at the cost of reranking latency. The `reranker_output_k` parameter (config key: `reranking.output_k`) controls how many reranked chunks are passed to context assembly.

| Parameter                         | Default | Min | Max  | Notes                                                               |
|-----------------------------------|---------|-----|------|---------------------------------------------------------------------|
| <code>reranking.enabled</code>    | true    | -   | -    | Set false to skip reranking (reduces accuracy ~20%)                 |
| <code>reranking.top_n</code>      | 40      | 5   | 200  | Input size to cross-encoder; higher = better recall, higher latency |
| <code>reranking.output_k</code>   | 20      | 3   | 50   | Reranked chunks passed to context assembly                          |
| <code>reranking.batch_size</code> | 8       | 1   | 32   | Parallel pairs per cross-encoder forward pass (GPU batch)           |
| <code>reranking.timeout_ms</code> | 200     | 50  | 2000 | Max reranking time before falling back to RRF order                 |

## 7. LLM Orchestration & Response Generation

The LLM Gateway is the final active processing stage before the response is returned to the user. It receives the packed context string and citation metadata from the Context Assembler, constructs a structured prompt, routes the request to the appropriate LLM provider, handles streaming and retries, and returns the generated response for validation.

### 7.1 Supported LLM Providers & Models

| Provider   | Models                                           | Context Window | Recommended For                          |
|------------|--------------------------------------------------|----------------|------------------------------------------|
| OpenAI     | GPT-4o, GPT-4o-mini, GPT-4-Turbo                 | 128k tokens    | Highest quality; general QA              |
| Anthropic  | Claude 3.5 Sonnet, Claude 3 Haiku, Claude 3 Opus | 200k tokens    | Long-document Q&A; complex reasoning     |
| Google     | Gemini 1.5 Pro, Gemini 1.5 Flash                 | 1M tokens      | Ultra-long context; multimodal queries   |
| Cohere     | Command R+, Command R                            | 128k tokens    | RAG-optimized; grounded generation       |
| Mistral    | Mistral Large 2, Mixtral 8x22B                   | 64k tokens     | European data residency; cost efficiency |
| Meta (OSS) | Llama 3.1 70B, Llama 3.1 405B                    | 128k tokens    | Self-hosted; no data egress              |
| NB-LLM-7B  | NeuralBridge fine-tuned Llama 3                  | 16k tokens     | Edge Lite mode; air-gapped deployments   |

### 7.2 Prompt Architecture

NeuralBridge uses a three-part prompt structure for all generation requests. This structure is templated in the LLM Gateway and cannot be bypassed by end users — this is a key safety and quality control measure.

| Prompt Section | Content                                                                         | Purpose                                                           |
|----------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------|
| System Prompt  | Role definition, citation requirements, format rules, hallucination prohibition | Set behavioral guidelines; constrains LLM to context-only answers |
| Context Block  | Top-K reranked chunks, each tagged with [SOURCE-N]                              | Provide factual basis for the answer                              |
| Query Block    | Normalized user query + any conversation history (multi-turn)                   | Define the question to be answered                                |

#### Default System Prompt Template:

```
You are a precise, citation-driven assistant. Answer the user's question using ONLY the information provided in the CONTEXT BLOCK below.

Rules:
1. Every factual claim MUST be followed by its citation: [SOURCE-N].
2. If the answer is not present in the context, reply: 'The provided documents do not contain sufficient information to answer this question.'
3. Do not use any external knowledge or training data facts.
4. Preserve technical terms, version numbers, and proper nouns exactly.
5. Format the answer in clear, structured prose. Use bullet points for lists.
```

### 7.3 Multi-Turn Conversation Mode

NeuralBridge supports multi-turn conversational RAG where previous query-answer pairs are included in subsequent prompts. The conversation history is managed by the Session Manager component, which

applies a sliding window to prevent context overflow. The window size is configurable via `llm.conversation_history_turns` (default: 5 turns).

In multi-turn mode, the query submitted to the retrieval layer is automatically reformulated using Conversational Query Rewriting (CQR) — a lightweight LLM call that resolves coreferences ('it', 'that system', 'the above method') and adds missing context from previous turns before the query is embedded and retrieved against.

## 7.4 Hallucination Detection & Confidence Scoring

The Response Validator applies two independent checks to every generated answer before it is returned to the caller:

- **Citation Coverage Check:** Every sentence in the answer is mapped to its cited source chunk. Sentences with no valid citation mapping trigger a hallucination flag.
- **Entailment Score:** An NLI (Natural Language Inference) model scores each answer sentence against its cited chunk to confirm the answer is logically entailed by the evidence. Score below 0.7 triggers a low-confidence flag.

| Confidence Level | Entailment Score | Citation Coverage | API Response Field | Behavior                                      |
|------------------|------------------|-------------------|--------------------|-----------------------------------------------|
| HIGH             | $\geq 0.85$      | $\geq 90\%$       | confidence: high   | Answer returned as-is                         |
| MEDIUM           | 0.70–0.84        | 75–89%            | confidence: medium | Answer returned with caveat note              |
| LOW              | 0.50–0.69        | 50–74%            | confidence: low    | Answer returned with strong disclaimer        |
| FAIL             | $< 0.50$         | $< 50\%$          | confidence: fail   | Fallback response returned; answer suppressed |

## 8. Multi-Modal Processing Module

As of version 3.0, NeuralBridge includes a Multi-Modal Processing Module (MPM) that extends the platform's capabilities beyond text to include images, tables, charts, audio transcripts, and structured data from within documents. The MPM operates as a pre-processing step inside the ingestion pipeline, enriching chunks with extracted non-text content.

### 8.1 Image & Chart Understanding

| Content Type         | Processing Method                            | Output                                   |
|----------------------|----------------------------------------------|------------------------------------------|
| Photographs          | CLIP embedding + alt-text generation via VLM | Text description chunk + image embedding |
| Charts/Graphs        | Chart-to-table extraction via ChartQA model  | Tabular data chunk + figure caption      |
| Diagrams/Flowcharts  | OCR + spatial relationship extraction        | Structured text description chunk        |
| Screenshots          | OCR + UI element detection                   | Text chunk with element labels           |
| Mathematical figures | LaTeX extraction via LatexOCR                | LaTeX string chunk + rendered text       |

### 8.2 Table Extraction

Tables in PDF documents are extracted using a two-step process: structural detection via the PaddleOCR table recognizer, followed by content extraction using pdfplumber's table extraction engine. Each table is stored as three separate chunk types: a prose summary chunk (for semantic retrieval), a TSV-format data chunk (for exact data lookup), and an original-layout chunk (for citation display).

### 8.3 Audio Transcription Pipeline

| Stage               | Tool                        | Notes                                                              |
|---------------------|-----------------------------|--------------------------------------------------------------------|
| Audio Extraction    | ffmpeg                      | Extracts audio track from MP4/MOV; converts to 16kHz WAV           |
| Diarization         | pyannote.audio              | Speaker separation — each speaker gets separate transcript stream  |
| Transcription       | Whisper v3 (large)          | High-accuracy transcription; 97.2% WER on English broadcast speech |
| Punctuation Restore | DeepMultilingualPunctuation | Restores sentence boundaries for chunking                          |
| Chunking            | Semantic Sentence           | Sentence-boundary chunking applied to transcript                   |
| Metadata            | Speaker ID + timestamp      | Appended to each chunk's metadata envelope                         |



## 9. Security Architecture & Access Controls

NeuralBridge was designed with a security-first architecture from version 1.0. All data in transit is encrypted with TLS 1.3. All data at rest uses AES-256-GCM encryption. The platform is SOC 2 Type II certified (as of October 2023) and GDPR-compliant by design.

### 9.1 Authentication & Authorization

| Mechanism       | Description                                              | Scope                           |
|-----------------|----------------------------------------------------------|---------------------------------|
| API Keys        | Static keys issued per integration; SHA-256 hashed in DB | Service-to-service calls        |
| JWT Tokens      | Short-lived (15min); RS256 signed; refresh via OAuth2    | End-user query API              |
| mTLS            | Mutual TLS between internal Octane Engine components     | Internal service mesh           |
| RBAC            | Role-Based Access Control with 5 built-in roles          | Corpus and query access control |
| SAML 2.0 / OIDC | Enterprise SSO integration (Okta, Azure AD, Google)      | User authentication federation  |

### 9.2 Built-in RBAC Roles

| Role        | Corpus Access      | Query Access     | Admin Actions   | Ingestion     |
|-------------|--------------------|------------------|-----------------|---------------|
| viewer      | Read (assigned)    | Query (assigned) | None            | None          |
| querier     | Read (assigned)    | Query (all)      | None            | None          |
| contributor | Read/write         | Query (all)      | None            | Upload docs   |
| admin       | Full               | Full             | Corpus settings | Full pipeline |
| super_admin | Full (all corpora) | Full (all)       | Full platform   | Full + delete |

### 9.3 Data Isolation — Multi-Tenancy

NeuralBridge supports strict multi-tenant data isolation. Each tenant's documents are stored in separate Qdrant collections (not shared collections with tenant filtering). This ensures that a Qdrant vulnerability cannot result in cross-tenant data leakage. The OpenSearch index uses separate index aliases per tenant with index-level access controls.

### 9.4 Audit Logging

| Event                   | Logged Fields                                                         |
|-------------------------|-----------------------------------------------------------------------|
| Query Submitted         | timestamp, user_id, tenant_id, query_text_hash, corpus_id, session_id |
| Document Ingested       | timestamp, user_id, doc_id, filename, file_size, chunking_strategy    |
| Document Deleted        | timestamp, user_id, doc_id, deletion_reason                           |
| API Key Created/Revoked | timestamp, admin_id, key_prefix, key_scope                            |
| Role Assignment Changed | timestamp, admin_id, target_user_id, old_role, new_role               |
| LLM Response Generated  | timestamp, query_id, model_used, confidence_level, token_count        |

## 10. Performance Benchmarks & Latency Targets

The following benchmarks were measured on the NeuralBridge Cloud (Standard tier) using a test corpus of 10 million chunks, GPT-4o-mini as the generation model, NB-Reranker-v3 as the cross-encoder, and NB-Embed-v2 as the embedding model. All measurements represent P95 latency across 10,000 test queries.

### 10.1 End-to-End Latency by Configuration

| Configuration                      | P50 Latency | P95 Latency | P99 Latency | Notes                              |
|------------------------------------|-------------|-------------|-------------|------------------------------------|
| Dense-only + No reranking          | 180ms       | 310ms       | 480ms       | Fastest; lowest accuracy           |
| Dense-only + Cross-encoder         | 260ms       | 420ms       | 620ms       | Good accuracy; no sparse           |
| Hybrid (RRF) + No reranking        | 210ms       | 360ms       | 540ms       | Balanced; recommended baseline     |
| Hybrid (RRF) + MiniLM-L6 reranking | 290ms       | 470ms       | 680ms       | Good accuracy+speed tradeoff       |
| Hybrid (RRF) + NB-Reranker-v3      | 380ms       | 590ms       | 820ms       | Default production config          |
| Hybrid (RRF) + Cohere Rerank 3.5   | 550ms       | 890ms       | 1200ms      | Highest accuracy; API latency adds |
| Edge Lite (quantized, CPU)         | 420ms       | 780ms       | 1100ms      | No GPU; small corpus only          |

### 10.2 Retrieval Quality Benchmarks

The following NDCG@10 and Recall@20 scores were measured using the NB-Eval-v2 evaluation suite, which consists of 4,800 human-annotated query-document relevance pairs across 6 domain categories: legal, finance, medical, software engineering, HR policy, and academic research.

| Retrieval Strategy           | NDCG@10 | Recall@20 | Precision@5 | MRR@10 |
|------------------------------|---------|-----------|-------------|--------|
| BM25 sparse only             | 0.341   | 0.612     | 0.298       | 0.388  |
| Dense only (NB-Embed-v2)     | 0.389   | 0.671     | 0.341       | 0.431  |
| Hybrid RRF (k=60)            | 0.421   | 0.718     | 0.374       | 0.468  |
| Hybrid + MiniLM-L6 reranking | 0.448   | 0.718     | 0.412       | 0.501  |
| Hybrid + NB-Reranker-v3      | 0.471   | 0.718     | 0.441       | 0.529  |
| Hybrid + Cohere Rerank 3.5   | 0.483   | 0.718     | 0.451       | 0.541  |
| Oracle (human upper bound)   | 0.612   | 0.891     | 0.578       | 0.671  |

**Note:** Recall@20 does not improve from Hybrid RRF to reranking stages because reranking does not change the candidate set — it only reorders it. The recall ceiling is set by the fusion step. To improve recall, increase retrieval.dense.top\_k and retrieval.sparse.top\_k, then increase reranking.top\_n accordingly.

### 10.3 Scalability Characteristics

| Corpus Size | Qdrant Memory (m=16) | Index Build Time | Query P95 (HNSW ef=128) |
|-------------|----------------------|------------------|-------------------------|
| 1M chunks   | ~6 GB                | ~12 min          | 8ms                     |
| 5M chunks   | ~30 GB               | ~55 min          | 14ms                    |

| Corpus Size | Qdrant Memory (m=16) | Index Build Time | Query P95 (HNSW ef=128) |
|-------------|----------------------|------------------|-------------------------|
| 10M chunks  | ~60 GB               | ~2.1 hrs         | 22ms                    |
| 50M chunks  | ~300 GB              | ~11 hrs          | 38ms                    |
| 100M chunks | ~600 GB              | ~22 hrs          | 61ms                    |

## 11. Deployment — Kubernetes & Helm

NeuralBridge is distributed as a Helm chart (chart version: 3.4.1, app version: 3.4.1) available from the official NeuralBridge Helm repository at <https://charts.neuralbridge.ai>. The chart deploys all Octane Engine components, Qdrant, OpenSearch, Redis (for session caching), and the NeuralBridge UI as a single, configurable release.

### 11.1 Prerequisites

| Requirement         | Minimum            | Recommended Production                  |
|---------------------|--------------------|-----------------------------------------|
| Kubernetes          | 1.25               | 1.28+                                   |
| Helm                | 3.10               | 3.12+                                   |
| Node CPU (engine)   | 4 vCPU             | 8 vCPU (or 16 for high-throughput)      |
| Node RAM (engine)   | 16 GB              | 32 GB                                   |
| Node GPU (optional) | NVIDIA T4 (16GB)   | NVIDIA A10G (24GB) or A100              |
| Qdrant node RAM     | Corpus size * 1.5x | 2x corpus vector size + 20% headroom    |
| Persistent storage  | 50 GB SSD          | 500 GB NVMe SSD (IOPS >= 3000)          |
| Kubernetes CNI      | Any                | Cilium (for network policy enforcement) |

### 11.2 Helm Install — Quick Start

```
# Add the NeuralBridge Helm repository
helm repo add neuralbridge https://charts.neuralbridge.ai
helm repo update

# Create a namespace
kubectl create namespace neuralbridge

# Install with default values (development mode)
helm install nb neuralbridge/neuralbridge \
  --namespace neuralbridge \
  --set global.mode=development \
  --set global.licenseKey=YOUR_LICENSE_KEY

# Install with custom values file (production)
helm install nb neuralbridge/neuralbridge \
  --namespace neuralbridge \
  -f values-production.yaml \
  --set global.licenseKey=YOUR_LICENSE_KEY

# Verify all pods are running
kubectl get pods -n neuralbridge
```

### 11.3 Key Helm Values

| Values Key              | Default        | Description                                                                                   |
|-------------------------|----------------|-----------------------------------------------------------------------------------------------|
| global.mode             | production     | development / production / edge-lite                                                          |
| global.licenseKey       | ""             | Required; obtain from <a href="https://neuralbridge.ai/licenses">neuralbridge.ai/licenses</a> |
| octane.replicaCount     | 3              | Octane Engine pod replicas                                                                    |
| octane.embedding.model  | NB-Embed-v2    | Embedding model identifier                                                                    |
| octane.reranker.model   | NB-Reranker-v3 | Cross-encoder model identifier                                                                |
| octane.llm.provider     | openai         | LLM provider: openai, anthropic, cohere, local                                                |
| octane.llm.model        | gpt-4o-mini    | Specific model name from provider                                                             |
| qdrant.persistence.size | 100Gi          | Persistent volume for Qdrant data                                                             |
| qdrant.replicaCount     | 1              | Qdrant replicas (use 3 for HA)                                                                |
| opensearch.replicas     | 1              | OpenSearch node count                                                                         |
| redis.auth.enabled      | true           | Enable Redis AUTH                                                                             |
| ingress.enabled         | true           | Enable Kubernetes ingress                                                                     |
| ingress.tls.enabled     | true           | TLS termination at ingress                                                                    |

## 12. API Reference

The NeuralBridge REST API follows OpenAPI 3.1 specification. The interactive API explorer is available at <https://api.neuralbridge.ai/docs>. All endpoints require authentication via Bearer token (JWT) or API Key in the Authorization header.

### 12.1 Core Query Endpoint

POST /v1/query

Request Body:

```
{
  "corpus_id": "corp_abc123",          // required
  "query": "What is the RRF k constant?", // required
  "session_id": "sess_xyz789",        // optional: for multi-turn
  "retrieval": {
    "top_k": 20,                      // override reranking.output_k
    "filters": {"file_type": "pdf"},  // metadata pre-filter
    "hybrid": true                    // override retrieval.hybrid.enabled
  },
  "llm": {
    "model": "gpt-4o-mini",           // override llm.model
    "max_tokens": 1024,
    "stream": false
  }
}
```

Response Body (200 OK):

```
{
  "query_id": "qry_def456",
  "answer": "The default RRF k constant in NeuralBridge is 60 [SOURCE-1]...",
  "confidence": "high",               // high | medium | low | fail
  "citations": [
    {"id": "SOURCE-1", "chunk_id": "...", "source_file": "nbp-trd.pdf",
     "page": 13, "text_preview": "The k constant controls..."},
  ],
  "retrieval_stats": {
    "dense_retrieved": 200, "sparse_retrieved": 200,
    "after_fusion": 100, "after_reranking": 20,
    "latency_ms": {"embedding": 22, "dense": 18, "sparse": 9,
                   "fusion": 2, "reranking": 87, "llm": 341, "total": 481}
  }
}
```

### 12.2 Corpus Management Endpoints

| Method | Endpoint                | Description                               |
|--------|-------------------------|-------------------------------------------|
| POST   | /v1/corpora             | Create a new corpus with configuration    |
| GET    | /v1/corpora             | List all corpora for authenticated tenant |
| GET    | /v1/corpora/{corpus_id} | Get corpus metadata and statistics        |
| DELETE | /v1/corpora/{corpus_id} | Delete corpus and all associated data     |

| Method | Endpoint                                   | Description                               |
|--------|--------------------------------------------|-------------------------------------------|
| POST   | /v1/corpora/{corpus_id}/documents          | Upload a document for ingestion           |
| GET    | /v1/corpora/{corpus_id}/documents          | List documents in corpus                  |
| DELETE | /v1/corpora/{corpus_id}/documents/{doc_id} | Delete a specific document                |
| GET    | /v1/corpora/{corpus_id}/stats              | Chunk count, index size, ingestion status |
| POST   | /v1/corpora/{corpus_id}/reindex            | Trigger full re-ingestion with new config |

## 13. Monitoring, Observability & Alerting

NeuralBridge exports metrics in Prometheus format from the Telemetry Exporter (port 9090). All inter-service calls are traced using OpenTelemetry with Jaeger as the default trace backend. Structured logs are emitted in JSON format to stdout for collection by Fluentd, Fluent Bit, or any Kubernetes-native log aggregator.

### 13.1 Key Prometheus Metrics

| Metric Name                        | Type      | Description                                             |
|------------------------------------|-----------|---------------------------------------------------------|
| nb_query_latency_seconds           | Histogram | End-to-end query latency; labels: corpus_id, confidence |
| nb_retrieval_dense_latency_ms      | Histogram | Dense HNSW search latency per query                     |
| nb_retrieval_sparse_latency_ms     | Histogram | BM25 search latency per query                           |
| nb_reranker_latency_ms             | Histogram | Cross-encoder batch latency                             |
| nb_llm_latency_ms                  | Histogram | LLM API call latency; labels: provider, model           |
| nb_queries_total                   | Counter   | Total queries received; labels: corpus_id, status       |
| nb_hallucination_flags_total       | Counter   | Queries where Response Validator flagged hallucination  |
| nb_confidence_low_total            | Counter   | Queries returning low or fail confidence                |
| nb_dense_retriever_full_scan_ratio | Gauge     | Fraction of dense queries triggering full scan          |
| nb_corpus_chunk_count              | Gauge     | Total indexed chunks per corpus                         |
| nb_ingestion_queue_depth           | Gauge     | Documents waiting in ingestion queue                    |
| nb_embedding_service_errors_total  | Counter   | Embedding service failure count                         |

### 13.2 Recommended Alerts

| Alert Name               | Condition                                      | Severity | Action                                            |
|--------------------------|------------------------------------------------|----------|---------------------------------------------------|
| HighQueryLatency         | nb_query_latency_seconds P95 > 2s for 5min     | Warning  | Check reranker load; scale reranker pods          |
| HallucinationRateHigh    | nb_hallucination_flags rate > 5% over 1hr      | Critical | Investigate LLM model changes; check context      |
| EmbeddingServiceErrors   | nb_embedding_service_errors rate > 1% per 1min | Critical | Check GPU availability; restart embedding pods    |
| DenseFullScanElevated    | nb_dense_retriever_full_scan_ratio > 0.3       | Warning  | Review metadata filter configurations             |
| IngestionQueueBacklog    | nb_ingestion_queue_depth > 1000 for 15min      | Warning  | Scale ingestion worker pool                       |
| LowConfidenceAnswersHigh | nb_confidence_low_total rate > 10% per 1hr     | Warning  | Review corpus quality; check retrieval parameters |



# 14. Known Limitations & Mitigations

No system is without limitations. This section documents known constraints of NeuralBridge 3.4.1, their root causes, and recommended mitigations. Transparency about limitations is a core NeuralBridge engineering value — teams should plan around these constraints rather than discover them in production.

| Limitation                   | Root Cause                                            | Impact | Mitigation                                                                  |
|------------------------------|-------------------------------------------------------|--------|-----------------------------------------------------------------------------|
| Embedding model mismatch     | Query and corpus must use identical embeddings        | High   | Remove short-term models; search changing embedding model; use N            |
| Context window overflow      | Very long chunks + many citations                     | Medium | Use hierarchical chunking with child size <= 256 tokens; set con            |
| Cold start latency           | GPU models (embedder, reranker) have JIT compilation  | Low    | Spin up warm queries first; request startup; use readinessProbe with        |
| Table retrieval accuracy     | Numerical tables often lose structure                 | Medium | Log table extraction mistakes; chunk type; add BM25 term boost for          |
| Cross-lingual retrieval      | BM25 sparse retrieval is language-specific            | Medium | Use language embeddings; upgrade to dense retrieval; filter metadata; disab |
| Very short queries           | Queries under 5 tokens produce low quality embeddings | Low    | Query Rewriter expands short queries; degrade LLM before embedd             |
| Real-time corpus updates     | HNSW index updates are eventually consistent          | Low    | Use ingestion webhook retrieval; apply 30 second delay                      |
| GPU memory (multiple models) | Running embedder + reranker + LLM on single GPU       | High   | Deploy embedder + reranker on separate GPU nodes; use CF                    |

## 15. Changelog & Version History

---

### 15.1 Version 3.4.1 — September 2024 (Current)

- **Fix:** Resolved race condition in Fusion Service when dense and sparse retrievers return simultaneously within 1ms of each other.
- **Fix:** NB-Reranker-v3 ONNX export now correctly handles batch sizes > 16 without truncation.
- **Perf:** Reduced Embedding Service cold start time from 8.2s to 3.1s via model pre-loading at container init.
- **Security:** Patched CVE-2024-3821 in the OpenSearch client dependency.

### 15.2 Version 3.4.0 — August 2024

- **Feature:** Added Cohere Rerank 3.5 as a supported cross-encoder option.
- **Feature:** Multi-Turn Conversation Mode (CQR) now supports up to 10 history turns (previously limited to 5).
- **Feature:** New Prometheus metric: nb\_dense\_retriever\_full\_scan\_ratio for filter monitoring.
- **Improvement:** RRF k constant now configurable at query-time via API parameter (previously deployment-time only).

### 15.3 Version 3.3.0 — June 2024

- **Feature:** NB-Embed-v2 released; set as default embedding model replacing bge-large-en-v1.5.
- **Feature:** Audio transcription pipeline added using Whisper v3 + pyannote.audio diarization.
- **Feature:** Edge Lite mode introduced for air-gapped and IoT deployments.
- **Breaking:** Minimum Kubernetes version raised from 1.22 to 1.25.

### 15.4 Version 3.0.0 — January 2024

- **Feature:** Multi-Modal Processing Module (MPM) introduced — image, chart, and table extraction.
- **Feature:** NB-Reranker-v3 released with DeBERTa-v3-Large base; replaces NB-Reranker-v2.
- **Feature:** Response Validator with NLI-based hallucination detection introduced.
- **Breaking:** API v0 endpoints removed. All clients must migrate to /v1/ API.

### 15.5 Version 2.8.0 — October 2023 (SOC 2 Type II milestone)

- **Security:** SOC 2 Type II certification achieved. Audit log retention extended to 2 years.
- **Feature:** SAML 2.0 and OIDC SSO integration added.
- **Feature:** Per-tenant Qdrant collection isolation implemented (replaces shared-collection + filter pattern).